

PSE-TRAVERSE-HORIZON-03

Null-Centered Horizon Geometry Collapse-Emission Duality und hypertoroidale Phasenraum-Kontrolle Formale Implementierungsspezifikation v0.3

Sebastian Klemm
Technische Ausarbeitung fuer Coding-Agent-Implementierung
Mai 2026

Zusammenfassung

Diese Spezifikation definiert eine domänenagnostische Horizontgeometrie fuer **pse-traverse**. Sie erweitert die bereits implementierte Projektions- und Finalisierungsschicht um ein nullzentriertes Koordinatensystem, einen hypertoroidalen Phasenraum, Phasenstrahlen, Projektionskegel, Horizon-Crossing-Gates und eine formale Collapse-Emission-Dualität. Das Ziel ist nicht die Modellierung astrophysischer Objekte, sondern die Operationalisierung eines hochdimensionalen Operatorbildes: interne Zustände werden relativ zu einem kanonischen Nullzentrum beobachtet, entlang deterministischer Phasenträger getaktet, durch Horizontfenster sichtbar gemacht, durch Kausal- und Projektionskegel beschränkt und nur bei dualer Rueckfuehrbarkeit als finalisierte Emission freigegeben.

Die Schicht ersetzt weder **pse-core** noch erzeugt sie **SemanticCrystal**. Sie erzeugt **HorizonChart**, **PhaseRay**, **ProjectionCone**, **HorizonCrossingReport**, **DualityReport**, **HorizonCertificate** und optional ein **FinalizedEmission**-Artefakt, das weiterhin nur ueber die bestehende PSE-Bridge in den Commit-Kern uebergeben werden darf.

Inhaltsverzeichnis

1	Status, Geltungsbereich und Ziel	2
1.1	Status	2
1.2	Ziel	2
1.3	Nicht-Zweck	2
2	Systemrolle innerhalb der Architektur	2
2.1	Schichtmodell	2
2.2	Harte Architekturregel	3
2.3	Konzeptuelle Position	3
3	Normative Begriffe	3
4	Neutralisierungspflicht	4
5	Kanonisches Datenmodell	4
5.1	Primitive	4
5.2	HorizonRunDescriptorV3	4

5.3	HorizonThresholdsV3	5
5.4	HorizonPoliciesV3	5
6	Kernobjekte	5
6.1	HorizonChart	5
6.2	PhaseCarrierSpecV3	6
6.3	PhaseRay	6
6.4	EventHorizonWindowV3	7
6.5	ProjectionCone	7
6.6	CausalAdmissibilityReport	7
6.7	CollapseEmissionDuality	8
6.8	HorizonCrossingReport	8
6.9	HorizonCertificate	9
7	Operatoren	9
7.1	NullCenterUnfold	9
7.2	PhaseRayLift	9
7.3	HorizonVisibility	9
7.4	ProjectionConeCheck	9
7.5	CausalOrderCheck	9
7.6	CollapseEmissionDualityCheck	10
7.7	HorizonCrossingGate	10
7.8	Superset Gate zu Projection v0.2	10
8	Referenzpipeline	10
8.1	Ablauf	10
8.2	Pseudocode	10
9	Outcomes und Fehlersemantik	11
9.1	Outcome-Typen	11
9.2	Fail-Closed-Regel	11
9.3	FailurePolicy-Auswahl	11
10	CLI-Schnittstelle	12
10.1	Subcommands	12
10.2	CLI-Anforderungen	12
11	Integration mit bestehenden Crates	12
11.1	Modulzuschnitt	12
11.2	Feature Flags	13
12	Determinismus und Canonicalization	13
13	Referenz-Zustandsmaschine	13
14	Testplan	14
14.1	Unit Tests	14
14.2	Integration Tests	14
14.3	Property Tests	14
15	Definition of Done	14
16	Coding-Agent-Auftrag	15

17 Schlussformel**16**

1 Status, Geltungsbereich und Ziel

1.1 Status

Dieses Dokument ist eine normative Nachfolgespezifikation zu PSE-TRAVERSE-PROJECTION-02. Es ist als Implementierungsgrundlage fuer Coding-Agenten formuliert. Es definiert konkrete Datentypen, Operatoren, Gates, Zustandsmaschinen, Reports, CLI-Schnittstellen, Tests und Definition-of-Done.

1.2 Ziel

PSE-TRAVERSE-HORIZON-03 fuehrt eine nullzentrierte Horizontgeometrie ein. Sie beantwortet die Frage:

Wann ist ein stabilisierter und projektionsfähiger Zustand im hypertoroidalen Phasenraum sichtbar, kausal zulässig, dual rueckfuehrbar und finalisierbar?

Die Zielpipeline lautet:

StableState → NullCenter → HorizonChart → PhaseRays → ProjectionCone → HorizonCrossing → DualityCh

1.3 Nicht-Zweck

Diese Spezifikation ist nicht:

- ein astrophysisches Modell;
- eine Behauptung ueber 3D-Objekte, schwarze Loecher, weisse Loecher oder den Urknall;
- ein Metaphysik-, Kosmologie- oder Ontologiemodul;
- ein Ersatz fuer `pse-core`;
- ein alternativer `SemanticCrystal`-Generator;
- ein probabilistischer Scheduler ohne Replay-Garantie;
- ein LLM-Wrapper;
- ein Zeta-, Trading-, Hardware-, Cyber-Defense- oder Quantum-Hardware-System.

Normative Trennung. Begriffe wie Horizont, Nullzentrum, Projektion, Kollaps und Emission sind in dieser Spezifikation rein operatorisch und topologisch zu lesen. Jede Implementierung MUSS diese Begriffe als Zustands-, Gate-, Projektions- und Replay-Konstrukte behandeln.

2 Systemrolle innerhalb der Architektur

2.1 Schichtmodell

Schicht	Rolle
<code>pse-core</code>	Evidence-, Gate-, Crystal- und Commit-Kern. Nur diese Schicht darf <code>SemanticCrystal</code> erzeugen.
<code>pse-traverse-core</code>	<code>ProblemSpec</code> , <code>FieldCube</code> , <code>DoFGraph</code> , <code>CollapsePlan</code> , <code>Candidate</code> .
<code>pse-traverse-signature</code>	Operator- und Signaturdiagnostik ueber strukturierte Räume.
<code>pse-traverse-dynamics</code>	Feldabsorption, Guidance, Kompression und <code>TransitionProof</code> .

pse-traverse-reactive	TriTemporalClock, ZeroPointCoupling, ResidualZero, ReactiveGate.
pse-traverse-projection v0.2	NullCenter, HypertorusScheduler, PhaseHorizon, ExclusionAxis, ExpansionAxis, FinalizationCore.
pse-traverse-horizon v0.3	Nullzentrierte Horizontgeometrie: HorizonChart, PhaseRay, ProjectionCone, CausalAdmissibility, CollapseEmissionDuality, HorizonCrossingReport.

2.2 Harte Architekturregel

Commit-Invariante. pse-traverse-horizon MUST NOT SemanticCrystal erzeugen. Die Schicht MAY FinalizedEmission, HorizonCertificate und HorizonGateReport erzeugen. Ein Crystal entsteht ausschließlich durch die bestehende PSE-Bridge und pse_core::macro_step.

2.3 Konzeptuelle Position

Die Horizontgeometrie macht projection v0.2 strenger. projection v0.2 prueft, ob eine Projektion finalisiert werden darf. horizon v0.3 prueft zusätzlich, ob die Projektion innerhalb einer nullzentrierten Phasenraum-Geometrie sichtbar, kausal geordnet, in einem Projektionskegel liegend und dual rueckfuehrbar ist.

3 Normative Begriffe

Begriff	Normative Bedeutung
NullCenter	Kanonischer Referenzkern eines Zustands. Content-addressed, idempotent, replayfähig.
HorizonChart	Vollständige nullzentrierte Koordinatisierung eines Phasenraums inklusive Träger, Fenster, Projektionskegel und Kausalordnung.
PhaseRay	Deterministische Trägerlinie vom NullCenter in den hypertoroidalen Phasenraum.
Hypertorus	Produkt zyklischer Phasenräume. Standardprofil in v0.3 ist T^4 , allgemein T^n .
EventHorizonWindow	Phasisch definiertes Sichtbarkeits- und Emissionsfenster.
ProjectionCone	Beschränkung der erlaubten Projektionsrichtungen und Zielräume.
CausalAdmissibility	Prüfung, ob ein Zustand die deklarierte Carrier- und Gate-Reihenfolge respektiert.
CollapseOperator	Verdichtet einen Zustand in Richtung NullCenter oder stabiler Kernrepräsentation.
EmissionOperator	Entfaltet einen Zustand aus NullCenter-/Kernrepräsentation in einen Zielraum.
CollapseEmissionDuality	Rundlaufpruefung, dass Verdichtung und Entfaltung konsistent zum gleichen NullCenter fuehren.
HorizonCrossing	Gate-geprüfter Übergang vom internen Zustand in eine externe, finalisierte Emission.

Attenuation	Verlust oder Dämpfung von Sichtbarkeit, Kohärenz oder Projektionstiefe entlang eines PhaseRay.
BoundaryTension	Residualsspannung an der Horizontgrenze; Gate-relevante Instabilitätsmetrik.

4 Neutralisierungspflicht

Nicht in den Kern	Neutraler Ersatz
Schwarzes Loch als astrophysisches Objekt	CollapseOperator, Absorb, Condense.
Weisses Loch als astrophysisches Objekt	EmissionOperator, RadialUnfolding, Project.
Urknall als Kosmologie	PhaseSpaceInitialization, NullCenterUnfold.
Singularität als physischer Ort	NullCenterObject, CanonicalOrigin.
Ereignishorizont als physische Wand	EventHorizonWindow, HorizonGate.
Zeitlinien als physische Weltlinien	PhaseRay, CarrierPath, SchedulerRay.
13D als ontologische Behauptung	Hochdimensionaler Zustands-, Feature-, Phasen- oder Hilbert-artiger Raum.
Kosmologische Sprache	Operatorische, topologische und replayfähige Implementierungssprache.

5 Kanonisches Datenmodell

5.1 Primitive

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = CanonicalNumber;

pub struct HorizonEvidenceRef {
  pub kind: String,
  pub address: Hash256,
  pub relation: String,
}
```

Gate-relevante numerische Werte MUESSEN als fixed-point, integer-rational oder anderweitig plattforminvariant implementiert werden. Plattform-Floats duerfen nicht in Audit-, Gate- oder Digest-Pfade eingehen.

5.2 HorizonRunDescriptorV3

```
pub struct HorizonRunDescriptorV3 {
  pub run_id: StableId,
  pub projection_v2_report_hash: Option<Hash256>,
  pub problem_spec_hash: Hash256,
  pub seed: u64,
  pub operator_versions: BTreeMap<String, String>,
  pub canonicalization_version: String,
  pub thresholds: HorizonThresholdsV3,
  pub policies: HorizonPoliciesV3,
```

```
}

```

5.3 HorizonThresholdsV3

```
pub struct HorizonThresholdsV3 {
  pub max_null_distance: Fixed,
  pub min_horizon_visibility: Fixed,
  pub max_boundary_tension: Fixed,
  pub max_attenuation: Fixed,
  pub min_causal_admissibility: Fixed,
  pub min_projection_cone_score: Fixed,
  pub min_duality_score: Fixed,
  pub max_roundtrip_delta: Fixed,
  pub min_crossing_score: Fixed,
  pub max_phase_jitter: Fixed,
  pub max_steps: u64,
}
```

5.4 HorizonPoliciesV3

```
pub struct HorizonPoliciesV3 {
  pub hypertorus_dimension: u8,           // default: 4
  pub carrier_policy: CarrierPolicyV3,
  pub horizon_window_policy: HorizonWindowPolicyV3,
  pub projection_cone_policy: ProjectionConePolicyV3,
  pub causal_policy: CausalPolicyV3,
  pub duality_policy: DualityPolicyV3,
  pub crossing_policy: CrossingPolicyV3,
  pub failure_policy_order: Vec<HorizonFailurePolicy>,
}

pub enum CarrierPolicyV3 {
  InheritFromProjectionV2,
  Explicit(Vec<PhaseCarrierSpecV3>),
  GenerateHypertorus { dimension: u8, rational_frequencies: Vec<RationalFixed> },
}

pub enum HorizonFailurePolicy {
  Hold,
  WaitForHorizon,
  RefineProjectionCone,
  SplitCarrier,
  MigrateCarrier,
  Recondense,
  RequireHumanReview,
  Abort,
}
```

6 Kernobjekte

6.1 HorizonChart

```
pub struct HorizonChart {
    pub chart_id: Hash256,
    pub rd_hash: Hash256,
    pub null_center_id: Hash256,
    pub hypertorus_dimension: u8,
    pub carrier_set_hash: Hash256,
    pub phase_ray_hashes: Vec<Hash256>,
    pub horizon_window_hashes: Vec<Hash256>,
    pub projection_cone_hash: Hash256,
    pub causal_order_hash: Hash256,
    pub evidence_refs: Vec<HorizonEvidenceRef>,
}
```

Chart-Invariante. HorizonChart MUST vollständig aus kanonischen Bytes berechnet werden. Traeger, Rays, Fenster und Kausalrelationen MUST vor Hashing deterministisch sortiert werden.

6.2 PhaseCarrierSpecV3

```
pub struct PhaseCarrierSpecV3 {
    pub carrier_id: StableId,
    pub frequency: RationalFixed,
    pub phase_offset: RationalFixed,
    pub role: PhaseCarrierRole,
    pub quantization: PhaseQuantizationV3,
}

pub enum PhaseCarrierRole {
    ExtrinsicProgress,
    IntrinsicResonance,
    HorizonVisibility,
    FinalizationReplay,
    Auxiliary(String),
}

pub enum PhaseQuantizationV3 {
    Continuous,
    KPolar { k: u32 },
    Tripolar { tau_low: Fixed, tau_high: Fixed },
    Quadrupolar { tau: Fixed },
}
```

6.3 PhaseRay

```
pub struct PhaseRay {
    pub ray_id: Hash256,
    pub origin_null_id: Hash256,
    pub carrier_id: StableId,
    pub epoch: u64,
    pub phase: Fixed,
    pub radial_depth: Fixed,
    pub attenuation: Fixed,
    pub boundary_tension: Fixed,
```



```
    pub gate_visibility: bool,  
    pub canonical_trace_hash: Hash256,  
}
```

6.4 EventHorizonWindowV3

```
pub struct EventHorizonWindowV3 {  
    pub window_id: Hash256,  
    pub carrier_id: StableId,  
    pub epoch_range: EpochRange,  
    pub phase_start: Fixed,  
    pub phase_end: Fixed,  
    pub min_amplitude: Fixed,  
    pub max_jitter: Fixed,  
    pub visibility_floor: Fixed,  
}  
  
pub struct HorizonWindowReportV3 {  
    pub report_id: Hash256,  
    pub window_id: Hash256,  
    pub ray_id: Hash256,  
    pub phase_in_window: bool,  
    pub epoch_in_range: bool,  
    pub amplitude_ok: bool,  
    pub jitter_ok: bool,  
    pub visibility_ok: bool,  
    pub passed: bool,  
}
```

6.5 ProjectionCone

```
pub struct ProjectionCone {  
    pub cone_id: Hash256,  
    pub null_center_id: Hash256,  
    pub allowed_target_kinds: Vec<String>,  
    pub allowed_layer_kinds: Vec<String>,  
    pub max_opening_angle: Fixed,  
    pub min_focus: Fixed,  
    pub max_dispersion: Fixed,  
    pub canonical_axis_hash: Hash256,  
}  
  
pub struct ProjectionConeReport {  
    pub report_id: Hash256,  
    pub cone_id: Hash256,  
    pub target_kind: String,  
    pub angle_score: Fixed,  
    pub focus_score: Fixed,  
    pub dispersion_score: Fixed,  
    pub passed: bool,  
}
```

6.6 CausalAdmissibilityReport

```

pub struct CausalAdmissibilityReport {
  pub report_id: Hash256,
  pub chart_id: Hash256,
  pub state_id: Hash256,
  pub declared_order_hash: Hash256,
  pub observed_order_hash: Hash256,
  pub admissibility_score: Fixed,
  pub violations: Vec<CausalViolation>,
  pub passed: bool,
}

pub struct CausalViolation {
  pub violation_id: Hash256,
  pub carrier_id: StableId,
  pub expected_before: StableId,
  pub observed_before: StableId,
  pub severity: Fixed,
}

```

6.7 CollapseEmissionDuality

```

pub struct CollapseEmissionDualitySpec {
  pub duality_id: StableId,
  pub collapse_operator_id: StableId,
  pub emission_operator_id: StableId,
  pub null_center_id: Hash256,
  pub max_roundtrip_delta: Fixed,
}

pub struct DualityReport {
  pub report_id: Hash256,
  pub duality_id: StableId,
  pub input_state_hash: Hash256,
  pub collapsed_hash: Hash256,
  pub emitted_hash: Hash256,
  pub returned_null_id: Hash256,
  pub roundtrip_delta: Fixed,
  pub duality_score: Fixed,
  pub passed: bool,
}

```

6.8 HorizonCrossingReport

```

pub struct HorizonCrossingReport {
  pub report_id: Hash256,
  pub chart_id: Hash256,
  pub state_id: Hash256,
  pub horizon_window_reports: Vec<HorizonWindowReportV3>,
  pub cone_report: ProjectionConeReport,
  pub causal_report: CausalAdmissibilityReport,
  pub duality_report: DualityReport,
  pub crossing_score: Fixed,
  pub crossing_allowed: bool,
  pub failure_policy: HorizonFailurePolicy,
  pub evidence_refs: Vec<HorizonEvidenceRef>,
}

```

```
}

```

6.9 HorizonCertificate

```
pub struct HorizonCertificate {
  pub certificate_id: Hash256,
  pub rd_hash: Hash256,
  pub chart_id: Hash256,
  pub crossing_report_hash: Hash256,
  pub projection_v2_certificate_hash: Option<Hash256>,
  pub finalized_emission_hash: Option<Hash256>,
  pub replay_hash: Hash256,
}
```

7 Operatoren

7.1 NullCenterUnfold

$$U_N : N \rightarrow H,$$

$$N \mapsto \text{HorizonChart}(N, T^n, C, W, \Gamma),$$

wobei N das NullCenter, T^n der hypertoroidale Phasenraum, C die Carrier-Menge, W die Horizontfenster und Γ die Kausalordnung bezeichnet.

Unfold-Invariante. NullCenterUnfold MUST deterministisch sein. Gleiches NullCenter, gleicher RunDescriptor und gleiche Operatorversionen MUESSEN denselben **HorizonChart** erzeugen.

7.2 PhaseRayLift

Der Operator PhaseRayLift erzeugt aus Carrierzustand und NullCenter eine Trägerlinie:

$$\text{Ray}_i(k) = (N, c_i, e_i(k), \tilde{\theta}_i(k), r_i(k), a_i(k), \tau_i(k)).$$

Dabei ist e_i die Epoche, $\tilde{\theta}_i$ die Phase modulo 2π , r_i die radiale Tiefe, a_i die Dämpfung und τ_i die BoundaryTension.

7.3 HorizonVisibility

Ein PhaseRay ist sichtbar, wenn alle deklarierten Fensterbedingungen erfuehlt sind:

$$G_{\text{visible}}(r, w) = 1 \iff \tilde{\theta}(r) \in [\alpha_w, \beta_w] \wedge e(r) \in E_w \wedge A(r) \geq \theta_A \wedge J(r) \leq \epsilon_J.$$

7.4 ProjectionConeCheck

$$G_{\text{cone}} = 1 \iff \text{angle_score} \geq \theta_{\text{angle}} \wedge \text{focus_score} \geq \theta_{\text{focus}} \wedge \text{dispersion_score} \leq \theta_{\text{dispersion}}.$$

7.5 CausalOrderCheck

$$G_{\text{causal}} = 1 \iff \text{score}(\text{observed_order}, \text{declared_order}) \geq \theta_{\text{causal}} \wedge \text{violations} = \emptyset.$$

7.6 CollapseEmissionDualityCheck

Die Dualitätsprüfung erzwingt, dass Verdichtung und Entfaltung auf dasselbe NullCenter rueckfuehrbar bleiben:

$$G_{dual} = 1 \iff d_N(N, \delta(Collapse(Emit(N)))) \leq \epsilon_N.$$

7.7 HorizonCrossingGate

$$G_{cross} = 1 \iff G_{visible} \wedge G_{cone} \wedge G_{causal} \wedge G_{dual} \wedge BoundaryTension \leq \theta_T \wedge Attenuation \leq \theta_A.$$

7.8 Superset Gate zu Projection v0.2

$$G_{v0.3} = G_{projection_v2} \wedge G_{cross} \wedge ReplayReady.$$

Falls $G_{v0.3} = 0$, MUSS das System einen Hold-, Wait-, Migration- oder Abort-Outcome erzeugen. Eine Finalisierung ohne $G_{v0.3} = 1$ ist unzulessig.

8 Referenzpipeline

8.1 Ablauf

1. Lade HorizonRunDescriptorV3 und Ausgangszustand.
2. Canonicalize Input und uebernehme oder berechne NullCenterObject.
3. Erzeuge HorizonChart durch NullCenterUnfold.
4. Erzeuge PhaseRays aus allen PhaseCarriern.
5. Evaluere EventHorizonWindowV3 fuer alle relevanten Rays.
6. Berechne ProjectionConeReport.
7. Berechne CausalAdmissibilityReport.
8. Berechne DualityReport.
9. Evaluere HorizonCrossingGate.
10. Verknuepfe mit ProjectionV2Outcome und HolisticMatrixGate.
11. Bei Pass: erzeuge oder annotiere FinalizedEmission.
12. Bei Fail: erzeuge HorizonHoldReport mit deterministischer HorizonFailurePolicy.
13. Erzeuge HorizonCertificate.
14. Pruefe Replay-Byte-Identität.

8.2 Pseudocode

```
pub fn run_horizon_v3(
  input: HorizonInput,
  rd: HorizonRunDescriptorV3,
) -> HorizonV3Outcome {
  let canonical = canonicalize(input, rd.canonicalization_version);
  let null_center = resolve_null_center(&canonical, &rd);

  let chart = unfold_horizon_chart(&null_center, &rd);
  let rays = lift_phase_rays(&chart, &rd);
  let windows = evaluate_horizon_windows(&rays, &chart, &rd);

  let cone = check_projection_cone(&canonical, &chart, &rd);
  let causal = check_causal_admissibility(&canonical, &chart, &rd);
  let dual = check_collapse_emission_duality(&canonical, &null_center, &rd);

  let crossing = horizon_crossing_gate(
    &chart, &windows, &cone, &causal, &dual, &rd
```

```

    );

    if !crossing.crossing_allowed {
        return HorizonV3Outcome::Hold(make_horizon_hold(
            chart, crossing, rd
        ));
    }

    let projection_v2 = run_or_load_projection_v2(&canonical, &rd);
    let gate = merge_projection_and_horizon_gates(&projection_v2, &crossing, &rd);

    if gate.passed {
        HorizonV3Outcome::Finalized(make_finalized_emission_v3(
            projection_v2, chart, crossing, rd
        ))
    } else {
        HorizonV3Outcome::Hold(make_horizon_hold(chart, crossing, rd))
    }
}

```

9 Outcomes und Fehlersemantik

9.1 Outcome-Typen

```

pub enum HorizonV3Outcome {
    Finalized(FinalizedEmissionV3),
    Hold(HorizonHoldReport),
    WaitForHorizon(HorizonHoldReport),
    RefineCone(HorizonHoldReport),
    NeedsCarrierMigration(HorizonHoldReport),
    Recondense(HorizonHoldReport),
    ProjectionOnly(ProjectionV2OutcomeRef),
    InvalidInput(HorizonDiagnostic),
    DeterminismViolation(HorizonDiagnostic),
}

```

9.2 Fail-Closed-Regel

Jede Unsicherheit in NullCenter, HorizonChart, Carrier, PhaseRay, ProjectionCone, Kausalordnung, Dualität, Gate oder Replay MUSS zu Hold oder härterem Fail führen. Unsicherer Zustand darf nicht finalisiert werden.

9.3 FailurePolicy-Auswahl

Die FailurePolicy MUSS deterministisch aus der ersten fehlgeschlagenen Pflichtklasse abgeleitet werden:

Fehlerklasse	Policy
Fenster geschlossen, sonst stabil	WaitForHorizon
Kegel verletzt	RefineProjectionCone
Carrier-Reihenfolge verletzt	MigrateCarrier
BoundaryTension zu hoch	Recondense
Dualitätsrunde verletzt	Recondense oder RequireHumanReview

Replay verletzt	Abort
Schema ungültig	InvalidInput

10 CLI-Schnittstelle

10.1 Subcommands

```
pse-traverse-horizon inspect <input.json>
pse-traverse-horizon chart <input.json> --rd horizon-rd.json --out chart.json
pse-traverse-horizon rays <chart.json> --out rays.json
pse-traverse-horizon crossing <input.json> --rd horizon-rd.json --out crossing.json
pse-traverse-horizon finalize <input.json> --rd horizon-rd.json --out final.json
pse-traverse-horizon replay <final-or-hold.json>
pse-traverse-horizon verify <horizon-certificate.json>
```

10.2 CLI-Anforderungen

- `inspect` MUST NullCenter, HorizonChart-Fähigkeit und erwartete Carrier anzeigen, ohne Finalisierung.
- `chart` MUST einen content-addressed `HorizonChart` erzeugen.
- `rays` MUST deterministische `PhaseRays` ausgeben.
- `crossing` MUST `HorizonWindow`, `Cone`, `Causal` und `Duality Reports` erzeugen.
- `finalize` MUST fehlschlagen, wenn $G_{v0.3} = 0$.
- `replay` MUST kanonische Byte-Identität prüfen.
- `verify` MUST Certificate-Hashes, Chart, CrossingReport und FinalizedEmission validieren.

11 Integration mit bestehenden Crates

11.1 Modulzuschnitt

```
crates/pse-traverse/src/horizon/
  mod.rs
  primitives.rs
  run_descriptor.rs
  horizon_chart.rs
  phase_carrier_v3.rs
  phase_ray.rs
  horizon_window.rs
  projection_cone.rs
  causal_admissibility.rs
  collapse_emission_duality.rs
  horizon_crossing.rs
  gate.rs
  outcome.rs
  certificate.rs
  replay.rs
  pipeline.rs

tools/pse-traverse-horizon-cli/src/main.rs
```

11.2 Feature Flags

Feature	Bedeutung
horizon	Aktiviert v0.3-Kern.
horizon-cli	Baut CLI-Tool.
horizon-projection-v2	Verknuepft Horizon-Gate mit Projection-v0.2-Pipeline.
horizon-pse-bridge	Erlaubt Weitergabe finalisierter Emissionen an bestehende PSE-Bridge.
horizon-adapters	Optionale DomainAdapter; Kern bleibt domänenneutral.

12 Determinismus und Canonicalization

Alle Maps MUESSEN als `BTreeMap` oder äquivalente geordnete Strukturen serialisiert werden. Vektoren ohne semantisch deklarierte Reihenfolge MUESSEN vor Hashing sortiert werden. Alle Reports, Certificates und Emissions MUESSEN JCS oder ein anderes dokumentiertes kanonisches Format verwenden.

Im Audit-Pfad verboten:

- Wall-clock timestamps;
- Plattform-Floats;
- ungeordnete `HashMap`-Iteration;
- OS-abhängige Pfadnormalisierung;
- nicht gespeicherte RNG-Entscheidungen;
- externe Netzwerkzustände;
- nicht versionsgestempelte Operatoren.

13 Referenz-Zustandsmaschine

Von	Nach	Bedingung
Idle	Loading	Input und RD vorhanden.
Loading	Canonicalizing	Dateien gelesen und Schema gueltig.
Canonicalizing	NullResolving	Canonical bytes erzeugt.
NullResolving	Charting	NullCenterObject vorhanden.
Charting	RayLifting	HorizonChart erzeugt.
RayLifting	Windowing	PhaseRays erzeugt.
Windowing	ConeChecking	Fensterreports erzeugt.
ConeChecking	CausalChecking	ProjectionConeReport erzeugt.
CausalChecking	DualityChecking	Kausalreport erzeugt.
DualityChecking	Crossing	DualityReport erzeugt.
Crossing	ProjectionV2Merge	HorizonCrossingGate bestanden.
Crossing	Holding	HorizonCrossingGate fehlgeschlagen.
ProjectionV2Merge	Finalizing	v0.2 und v0.3 Gate bestanden.
ProjectionV2Merge	Holding	kombiniertes Gate fehlgeschlagen.
Finalizing	Certified	FinalizedEmission und Certificate erzeugt.

Holding	Certified	HoldReport und Certificate erzeugt.
Certified	Replayed	Replay erfolgreich.
Jede Phase	Failed	Schema-, Determinismus- oder Invarianzverletzung.

14 Testplan

14.1 Unit Tests

1. horizon_chart_is_content_addressed
2. horizon_chart_is_deterministic
3. phase_ray_hash_changes_with_epoch
4. phase_ray_sorts_carriers_before_hash
5. horizon_window_passes_inside_interval
6. horizon_window_fails_outside_interval
7. projection_cone_fails_on_dispersion
8. causal_admissibility_detects_order_violation
9. duality_roundtrip_passes_same_nullcenter
10. duality_roundtrip_fails_changed_nullcenter
11. horizon_crossing_requires_all_gates
12. gate_fail_never_finalizes
13. wait_for_horizon_selected_when_only_window_closed
14. certificate_changes_when_crossing_changes
15. replay_detects_modified_chart

14.2 Integration Tests

1. cli_chart_writes_horizon_chart
2. cli_rays_byte_identical_across_runs
3. cli_crossing_writes_hold_or_pass
4. cli_finalize_refuses_failed_horizon
5. cli_finalize_passes_with_projection_v2_gate
6. cli_replay_byte_identical
7. cli_verify_detects_hash_tamper

14.3 Property Tests

1. Permutierte Carrier-Eingaben erzeugen identische `HorizonChart`-IDs.
2. Verschärfung einer Einzelschwelle kann ein Gate nur gleich lassen oder von Pass nach Fail kippen.
3. `HorizonCertificate`-ID ist Funktion aller referenzierten Hashes.
4. `FinalizedEmission` ist ohne `HorizonCrossingReport` ungültig.
5. `SemanticCrystal` wird in keinem Horizon-Modul konstruiert.

15 Definition of Done

Eine Implementierung von PSE-TRAVERSE-HORIZON-03 ist abgeschlossen, wenn:

1. `cargo test -workspace` besteht.
2. `cargo clippy -workspace - -D warnings` besteht.
3. `cargo doc -workspace --no-deps` besteht.
4. Horizon-Kern kompiliert ohne Netzwerkabhängigkeit.

5. Alle oeffentlichen Typen sind dokumentiert.
6. Alle Gate-Pfade sind fail-closed.
7. Keine Plattform-Floats treten in Audit-, Gate- oder Digest-Pfaden auf.
8. `HorizonChart` ist content-addressed und replayfähig.
9. `PhaseRays` sind deterministisch und carriersortiert.
10. `ProjectionConeReport`, `CausalAdmissibilityReport` und `DualityReport` sind gate-wirksam.
11. `HorizonCrossingGate` verhindert Finalisierung bei einem einzigen fehlgeschlagenen Pflicht-gate.
12. `WaitForHorizon` wird erzeugt, wenn ausschließlich das Phasenfenster geschlossen ist.
13. `FinalizedEmissionV3` wird nur bei $G_{v0.3} = 1$ erzeugt.
14. `HorizonCertificate` erkennt veränderte RD, Charts, Reports oder Emissions.
15. CLI `inspect`, `chart`, `rays`, `crossing`, `finalize`, `replay`, `verify` funktioniert end-to-end.
16. Zwei identische Runs erzeugen byte-identische Reports.
17. PSE-Bridge bleibt einziger Pfad zu `SemanticCrystal`.

16 Coding-Agent-Auftrag

PATCH-ID: PSE-TRAVERSE-HORIZON-03

Ziel:

Implementiere eine optionale nullzentrierte Horizontgeometrie fuer `pse-traverse`. Die Schicht erweitert `Projection v0.2` um `HorizonChart`, `PhaseRay`, `ProjectionCone`, `CausalAdmissibility`, `CollapseEmissionDuality`, `HorizonCrossingGate` und `HorizonCertificate`. Sie finalisiert nur, wenn `Projection-v0.2-Gates` und `Horizon-v0.3-Gates` gleichzeitig bestehen.

MUST:

- neue Module unter `crates/pse-traverse/src/horizon/` anlegen;
- `HorizonRunDescriptorV3`, `HorizonChart`, `PhaseRay`, `ProjectionCone`, `CausalAdmissibilityReport`, `DualityReport`, `HorizonCrossingReport`, `HorizonCertificate` implementieren;
- alle Reports und Certificates canonical serialisieren;
- Gate- und Audit-Pfade floatfrei halten;
- fail-closed gating erzwingen;
- `FinalizedEmissionV3` nur bei `G_v0.3` erzeugen;
- `HorizonHoldReport` bei Gate-Fail erzeugen;
- CLI-Tool mit `inspect/chart/rays/crossing/finalize/replay/verify` bereitstellen;
- Tests gemaess Testplan liefern;
- bestehende `Projection-v0.2-Pipeline` integrieren, nicht duplizieren;
- PSE `SemanticCrystal`-Erzeugung ausschliesslich bestehender PSE-Bridge ueberlassen.

MUST NOT:

- astrophysische, kosmologische, Riemann-, Trading-, Quantum-Hardware- oder Domain-Spezialsemantik in den Kern einbauen;
- nichtdeterministische RNG- oder Wall-clock-Werte in Digests schreiben;
- `HashMap`-Iteration in kanonischen Pfaden verwenden;
- Fliesskommazahlen fuer Gate-/Audit-Hashes verwenden;
- bei fehlgeschlagenem Gate finalisieren;
- `SemanticCrystal` in Horizon-Modulen erzeugen.

SHOULD:

- `BTreeMap/BTreeSet` fuer kanonische Strukturen verwenden;
- Feature Flags `horizon`, `horizon-cli`, `horizon-projection-v2`, `horizon-pse-bridge` und `horizon-adapters` anlegen;
- golden Replay Fixtures fuer `Pass`, `Hold`, `WaitForHorizon` und `Tamper` liefern.

17 Schlussformel

$$Finalize_{v0.3}(s, RD) = Emit(s)$$

nur wenn:

NullCenter(s) kanonisch ist,
HorizonChart(NullCenter(s), RD) replayfähig ist,
PhaseRays in gültigen *HorizonWindows* sichtbar sind,
ProjectionCone(s) den Zielraum erlaubt,
CausalOrder(s) admissible ist,
CollapseEmissionDuality(s) zum selben *NullCenter* zurueckfuehrt,
ProjectionV2Gate(s) = 1,
HorizonCrossingGate(s) = 1,
Replay(s) byte-identisch möglich ist.